

Sequential searching:

This is the simplest of all searching techniques. In this technique, an unordered search table is scanned from its first record until the desired record is found.

The search table is organized either in an array or as linked list. We will consider the array representation.

The C function for sequential search is given below:

```
int seq_search(int a[], int n, int key)
{
    int i;
    for(i=0; i<n; i++)
    {
        if(key ==a[i])
            return i;
    }
    return(-1);
}
```

In this function a[] is the array of records (in this case integers), n is the size, and key is the key to be searched.

If the key is found, then it will return the index i of that record. If the key is not found then it will return -1.

Efficiency of Sequential Searching:

Let us examine the number of comparisons made by a sequential search in searching for a given key. Let the size of the table be n. The number of comparisons depends on where the record with the argument key appears in the table.

If the record is the first one in the table only one comparison is performed, if the record is the last one in the table, n comparisons are needed. If it is equally likely for the key to appear at any given table position, a successful search will take (on the average) $(n+1)/2$ comparisons. An unsuccessful search will take n comparisons. In any case, the no. of comparisons is $O(n)$.